

DATABASE DESIGN & IMPLEMENTATION · WEEK 2

MoMo SMS Analytics

Database Design Document

Author	Clive Tanaka Mushipe
Email	c.mushipe@alustudent.com
Team	Solo
Database	momo_sms_db
Platform	MySQL 9.7.1 · InnoDB · utf8mb4
Repository	Database_Design_and_Implementation

Every query result and error message reproduced in this document was captured from a live run against the database built by `database/database_setup.sql`.

Contents

1. Introduction and scope

2. Entity Relationship Diagram

3. Design rationale

4. Entities and relationship cardinality

5. Data dictionary

6. Referential integrity and constraints

7. Index strategy

8. Security and accuracy rules, with evidence

9. Sample queries, with results

10. CRUD operation testing

11. JSON data modelling and SQL → JSON mapping

12. The dashboard, and why there is no frontend code

13. Reproducing this build

1. Introduction and scope

The MoMo SMS Analytics system turns an XML export of MTN Mobile Money SMS messages into queryable, auditable transaction data. This document specifies the database that sits at the centre of it: the entity model, the schema as built, the rules that protect data quality, and evidence that all of it works.

The schema is implemented in `database/database_setup.sql`, a single idempotent script that drops and rebuilds the database, creates seven tables with their constraints, indexes, triggers and views, provisions two least-privilege accounts, and loads realistic seed data. It runs to completion with no errors and no warnings.

On the sample data. Twenty-five of the thirty-two seeded transactions are derived from the course dataset at `data/raw/modified_sms_v2.xml`: their amounts, dates, phone numbers, merchant till codes and SMS bodies are the values in that file, and the party names were extracted from the message text. The remaining seven are synthetic and marked as such in the script; they exist to cover the five categories the sample does not exercise (`CASH_IN` , `CASH_OUT` , `BANK_DEPOSIT` , `UTILITY_PAYMENT` , `INTERNATIONAL_IN` , `REVERSAL`) so that every category and every constraint keeps test coverage. No real customer data appears anywhere in this repository.

The schema was tested against that dataset, and an early version failed it. The first design keyed `users` on a mandatory, strictly formatted phone number. That schema could not store a single row of the course data: subscriber numbers arrive as `0789876543` rather than `E.164`, and roughly a third of the counterparties, merchant tills such as `MTN:MoMoPay:Kigali_Mart` and service endpoints such as `MTN:Airtime` , have no phone number at all. Section 3 explains the change that fixed it.

What was built

Artefact	Path	Contents
ERD	<code>docs/erd_diagram.drawio</code> / <code>.svg</code> / <code>.png</code>	Crow's-foot ERD; the <code>.drawio</code> file opens and edits in diagrams.net
Schema	<code>database/database_setup.sql</code>	DDL, constraints, indexes, triggers, views, grants, seed DML
Queries	<code>database/sample_queries.sql</code>	10 queries covering joins, aggregates, a window function and FULLTEXT
CRUD tests	<code>database/crud_tests.sql</code>	Create / Read / Update / Delete with before-and-after state
Rule tests	<code>database/security_rules_demo.sql</code>	13 security and accuracy rules, each demonstrated against the engine
JSON	<code>examples/json_schemas.json</code>	draft-07 schemas plus examples generated from the live database

2. Entity Relationship Diagram



The two many-to-many relationships. `users` M:N `transactions` is resolved by `transaction_participants`, which qualifies every link with a `role`. `transactions` M:N `tags` is resolved by `transaction_tags`, whose composite primary key (`transaction_id`, `tag_id`) also carries the relationship's own attributes, `confidence` and `tagged_by`.

3. Design rationale

The design starts from one observation about the source data: a MoMo SMS does not describe a tidy pair of customers. An incoming transfer names a sender and a receiver; an airtime purchase names only the buyer; a bank deposit names an institution; a failed message may name nobody at all. Any schema that hard-codes `sender_id` and `receiver_id` onto the transaction row inherits that irregularity as a column full of NULLs.

So the central decision was to treat participation as a relationship in its own right. `users` and `transactions` stand in a genuine many-to-many relationship (a user takes part in many transactions, and a transaction involves several users) and `transaction_participants` resolves it, qualifying each link with a `role`. This removes the nullable-column problem, lets “everything user X did” run as one indexed query instead of a UNION of two, and leaves room to add a third role later without a migration. A second many-to-many relationship, between transactions and tags, is resolved by `transaction_tags`, whose composite primary key carries the relationship's own attributes: `confidence` and `tagged_by` belong to neither entity alone.

Categories live in a lookup table rather than an ENUM on the transaction, because a category carries its own attributes, a balance `direction` and a description, and because MTN adds product types without consulting our schema. Money is `DECIMAL(15,2)` and never `FLOAT`: a rounding error in a stored balance is a defect that compounds silently. `system_logs.transaction_id` is deliberately nullable, because a message that fails during parsing never becomes a transaction, and that failure is precisely the record an engineer needs.

The second decision came from testing the schema against the real dataset rather than reasoning about it. The first version keyed `users` on a mandatory phone number constrained to E.164, and it could not store a single row of the course data: subscriber numbers arrive in local `07...` form, and about a third of the counterparties are merchant tills or service endpoints with no number at all. Keying the table on `party_ref` instead, a canonical identifier holding either the normalised MSISDN or the service code makes every party storable, while `phone_number` becomes a nullable attribute still validated whenever it is present. A `BEFORE INSERT` trigger rewrites `07...` to `+250...` before the constraint is evaluated, so raw source values load unmodified and one subscriber cannot be stored twice under two spellings. The guarantee is kept and the data fits; the original design would have required discarding half of it.

Correctness is pushed down into the engine wherever it will go. A `UNIQUE` index on the SHA-256 of the message body makes re-running the ETL idempotent. `CHECK` constraints reject non-positive amounts and malformed phone numbers. Rules that span rows or need `NOW()`, namely no self-transfers, no future dates and mandatory audit of amount changes, are triggers, because a `CHECK` constraint cannot express them. A documented rule is a suggestion; an enforced rule is a guarantee.

483 words

4. Entities and relationship cardinality

Relationship	Card.	Implemented by	Reading
transaction_categories → transactions	1 : M	transactions.category_id	One category classifies many transactions; each transaction has exactly one category.
users ↔ transactions	M : N	transaction_participants junction	A user takes part in many transactions; a transaction involves several users. Each link is qualified by a role.
users → transaction_participants	1 : M	transaction_participants.user_id	One user has many participation rows.
transactions → transaction_participants	1 : M	transaction_participants.transaction_id	One transaction has up to two participation rows (one per role).
transactions ↔ tags	M : N	transaction_tags junction	A transaction carries many tags; a tag is applied to many transactions.
transactions → system_logs	1 : M	system_logs.transaction_id (nullable)	One transaction has many log entries. A log entry may belong to none.

Why the participants junction, and not two columns. With `sender_id` and `receiver_id` on the transaction row, an airtime purchase leaves `receiver_id` NULL, a failed message leaves both NULL, and the question “show me everything user X did” becomes `WHERE sender_id = X OR receiver_id = X`, a predicate no single index can serve. The junction makes each participation one row, one index entry, one uniform query. Query 3 and Query 4 in section 9 demonstrate both directions.

5. Data dictionary

Generated directly from `information_schema`, so it describes the database that actually exists. Every description below is the `COMMENT` stored on the column itself, the documentation lives in the schema, not only in this document.

users Core entity 27 rows seeded

Every party that can appear in a MoMo transaction: customers, merchants behind a payment code, mobile-money agents, partner banks, and the MTN system account used for airtime.

Key	Column	Type	Null	Default	Description
PK	user_id	int unsigned	NOT NULL	AUTO_INCREMENT	Surrogate primary key. Internal only, never exposed in the SMS text.
UK	party_ref	varchar(64)	NOT NULL	n/a	Canonical identifier and natural key. Either a normalised E.164 MSISDN (" +250788123456") or a service code ("MTN:MoMoPay:Kigali_Mart"). Always present.
UK	phone_number	varchar(16)	NULL	n/a	E.164 MSISDN when this party is a phone subscriber. NULL for merchant tills and service endpoints, which have no number.
	full_name	varchar(120)	NULL	n/a	Display name as it appears in the SMS body. NULL when the source never names the party.
	user_type	enum('customer','merchant','agent','bank','system')	NOT NULL	customer	Party classification. Drives how the counterparty is displayed in the dashboard.
	is_verified	tinyint(1)	NOT NULL	0	Whether the wallet has completed KYC verification.
	national_id	varchar(32)	NULL	n/a	Optional KYC reference. NULL for counterparties we only know from an SMS. Never exported to the dashboard.
	account_status	enum('active','suspended','closed')	NOT NULL	active	Lifecycle state of the MoMo wallet. Suspended/closed wallets are excluded from active-user analytics.
	first_seen_at	datetime	NOT NULL	CURRENT_TIMESTAMP	Timestamp of the earliest SMS in which this party appeared. Used for cohort analysis.
	created_at	datetime	NOT NULL	CURRENT_TIMESTAMP	Row insertion time (audit).
	updated_at	datetime	NOT NULL	CURRENT_TIMESTAMP	Row last-modified time (audit).

transaction_categories Lookup 10 rows seeded

The transaction taxonomy. Seeded from the canonical category list in `etl/config.py` so the database and the Python pipeline cannot drift apart.

Key	Column	Type	Null	Default	Description
PK	category_id	smallint unsigned	NOT NULL	AUTO_INCREMENT	Surrogate primary key.
UK	category_code	varchar(40)	NOT NULL	n/a	Machine-readable code used by the ETL, e.g. "INCOMING_TRANSFER". Matches CATEGORIES in <code>etl/config.py</code> .
UK	category_name	varchar(80)	NOT NULL	n/a	Human-readable label shown on the dashboard, e.g. "Incoming Money Transfer".
	direction	enum('credit','debit','neutral')	NOT NULL	n/a	Effect on the wallet balance: credit increases it, debit decreases it, neutral for reversals and informational messages.
	description	varchar(255)	NOT NULL	n/a	Plain-English definition of the category, used as dashboard tooltip text and in the data dictionary.
	is_active	tinyint(1)	NOT NULL	1	Soft-delete flag. Retired categories stay for historical rows but are hidden from new classification.
	created_at	datetime	NOT NULL	CURRENT_TIMESTAMP	Row insertion time (audit).

transactions Core entity 32 rows seeded

The central fact table: one row per MoMo SMS that represents a financial event. Holds no sender/receiver columns by design.

Key	Column	Type	Null	Default	Description
PK	transaction_id	bigint unsigned	NOT NULL	AUTO_INCREMENT	Surrogate primary key. BIGINT because SMS volume grows without bound.
UK	external_txn_ref	varchar(40)	NOT NULL	n/a	MTN transaction ID parsed from the SMS body (e.g. "TXN001"). Unique per transaction.
FK	category_id	smallint unsigned	NOT NULL	n/a	FK -> transaction_categories.category_id. Assigned by etl/categorize.py.
	amount	decimal(15,2)	NOT NULL	n/a	Principal amount in RWF. DECIMAL (not FLOAT) so currency arithmetic is exact.
	fee	decimal(15,2)	NOT NULL	0.00	Transaction fee charged by MTN, in RWF. Zero when the message reports no fee.
	balance_after	decimal(15,2)	NULL	n/a	Wallet balance after the transaction, when the SMS reports it. NULL if absent from the message.
	currency	char(3)	NOT NULL	RWF	ISO-4217 currency code. Constant RWF today; present so the schema survives multi-country expansion.
	transaction_date	datetime	NOT NULL	n/a	When the transaction occurred, parsed from the SMS. Primary time axis for all reporting.
	status	enum('pending','completed','failed','reversed')	NOT NULL	completed	Settlement state. Most parsed SMS messages report a completed transaction.
	channel	enum('sms','ussd','app','api')	NOT NULL	sms	How the transaction reached us. Always "sms" for the XML export pipeline.
	raw_sms_body	text	NOT NULL	n/a	Verbatim SMS text. Retained for auditability, re-parsing after ETL fixes, and FULLTEXT search.
UK	sms_hash	char(64)	NOT NULL	n/a	SHA-256 of the raw SMS body. UNIQUE - this is what makes the ETL idempotent on re-runs.
	notes	varchar(500)	NULL	n/a	Free-text annotation added by an analyst during review. Never written by the ETL.
	processed_at	datetime	NOT NULL	CURRENT_TIMESTAMP	When the ETL wrote this row.
	created_at	datetime	NOT NULL	CURRENT_TIMESTAMP	Row insertion time (audit).
	updated_at	datetime	NOT NULL	CURRENT_TIMESTAMP	Row last-modified time (audit).

transaction_participants Junction (M:N) 64 rows seeded

Resolves users M:N transactions. Each row links one user to one transaction in a specific role.

Key	Column	Type	Null	Default	Description
PK	participation_id	bigint unsigned	NOT NULL	AUTO_INCREMENT	Surrogate primary key for the junction row.
FK	transaction_id	bigint unsigned	NOT NULL	n/a	FK -> transactions.transaction_id.
FK	user_id	int unsigned	NOT NULL	n/a	FK -> users.user_id.
UK	role	enum('sender','receiver')	NOT NULL	n/a	Side of the transaction this user is on. Extensible to agent/biller later.
	party_label	varchar(120)	NULL	n/a	Identifier exactly as it appeared in this SMS, kept even if the canonical users row is later corrected.
	created_at	datetime	NOT NULL	CURRENT_TIMESTAMP	Row insertion time (audit).

tags

Lookup

10 rows seeded

Reusable labels: analytics buckets, data-quality flags, spending themes.

Key	Column	Type	Null	Default	Description
PK	tag_id	smallint unsigned	NOT NULL	AUTO_INCREMENT	Surrogate primary key.
UK	tag_name	varchar(50)	NOT NULL	n/a	Lowercase snake_case label, e.g. "high_value". Unique across the system.
	tag_type	enum('analytics','data_quality','user_defined')	NOT NULL	analytics	Origin of the tag: derived by analytics rules, raised by data-quality checks, or added manually.
	description	varchar(255)	NULL	n/a	What the tag means and when it is applied.
	created_at	datetime	NOT NULL	CURRENT_TIMESTAMP	Row insertion time (audit).

transaction_tags

Junction (M:N)

28 rows seeded

Resolves transactions M:N tags. Composite primary key, and carries the relationship's own attributes.

Key	Column	Type	Null	Default	Description
PFK	transaction_id	bigint unsigned	NOT NULL	n/a	FK -> transactions.transaction_id. Part of the composite PK.
PFK	tag_id	smallint unsigned	NOT NULL	n/a	FK -> tags.tag_id. Part of the composite PK.
	confidence	decimal(3,2)	NOT NULL	1.00	How certain the tagging rule is, 0.00-1.00. An attribute of the RELATIONSHIP, not of either entity. Manual tags are 1.00; heuristics may be lower.
	tagged_by	enum('etl','analyst','rule_engine')	NOT NULL	etl	What applied the tag, for provenance.
	tagged_at	datetime	NOT NULL	CURRENT_TIMESTAMP	When the tag was applied.

system_logs

Core entity

18 rows seeded

Operational audit trail for the ETL pipeline and for changes made to stored transactions.

Key	Column	Type	Null	Default	Description
PK	log_id	bigint unsigned	NOT NULL	AUTO_INCREMENT	Surrogate primary key.
FK	transaction_id	bigint unsigned	NULL	n/a	FK -> transactions.transaction_id. NULL when the message never became a transaction (e.g. a parse failure).
	stage	enum('parse','clean','categorize','load','export','audit')	NOT NULL	n/a	Pipeline stage that emitted the entry. Mirrors the modules in etl/.
	event_type	varchar(80)	NOT NULL	n/a	Machine-readable event name, e.g. PARSE_SUCCESS, DB_INSERT_FAIL. Indexed, so it filters far better than the free-text message.
	log_level	enum('DEBUG','INFO','WARNING','ERROR','CRITICAL')	NOT NULL	INFO	Severity. ERROR and above drive the dead-letter queue and the triage query.
	message	varchar(500)	NOT NULL	n/a	Human-readable description of what happened.
	source_file	varchar(255)	NULL	n/a	Input file being processed, e.g. "data/raw/modified_sms_v2.xml".
	record_ref	varchar(100)	NULL	n/a	Pointer to the offending record (XML index or transaction reference) for dead-letter replay.
	records_affected	int unsigned	NOT NULL	0	Row count touched by the stage. Used to report throughput per run.
	ip_address	varchar(45)	NULL	n/a	Host that performed the action. VARCHAR(45) so an IPv6 address fits as well as IPv4.
	created_at	datetime	NOT NULL	CURRENT_TIMESTAMP	When the entry was written.

6. Referential integrity and constraints

6.1 Foreign keys

The `ON DELETE` rule differs per relationship, and each choice is deliberate.

Child column	References	ON DELETE	ON UPDATE
system_logs.transaction_id	transactions.transaction_id	SET NULL	CASCADE
transaction_participants.transaction_id	transactions.transaction_id	CASCADE	CASCADE
transaction_participants.user_id	users.user_id	RESTRICT	CASCADE
transaction_tags.tag_id	tags.tag_id	CASCADE	CASCADE
transaction_tags.transaction_id	transactions.transaction_id	CASCADE	CASCADE
transactions.category_id	transaction_categories.category_id	RESTRICT	CASCADE

- **RESTRICT** on category and on user: financial history must never be orphaned. A category or a customer that has transacted cannot be deleted.
- **CASCADE** on the junction rows: a participation row has no meaning without its transaction, so it goes when the transaction goes.
- **SET NULL** on `system_logs`: deleting a transaction must not destroy the audit trail explaining what happened to it. The log survives with its pointer detached.

6.2 CHECK constraints

[illegible]

A conditional constraint is still a guarantee. `chk_users_phone_format` reads `phone_number` IS NULL OR `phone_number` REGEXP '`^\+250[0-9]{9}$`'. The IS NULL branch is not a loophole: it says that a party may have no phone number, a merchant till does not, while any number that *is* stored must be well formed. The alternative, relaxing the pattern to accept local `07...` format as well, would genuinely weaken the rule, because the same subscriber could then be stored twice under two spellings and no index could tell that they were the same person.

Normalisation happens before validation rather than instead of it. `trg_users_normalise_input` rewrites `07...` to `+250...`, and MySQL evaluates `BEFORE INSERT` triggers before `CHECK` constraints, so the constraint sees the canonical value. Raw source data loads unmodified and the strict rule still holds.

7. Index strategy

Primary keys and UNIQUE constraints create indexes implicitly; the table below lists everything, with the query each additional index was created to serve. An index that serves no query is pure write overhead, so each one is justified.

Table	Index	Column(s)	Type	Why it exists
system_logs	idx_logs_event_type	event_type	BTREE	Enforces a uniqueness constraint.
system_logs	idx_logs_level_created	log_level,created_at	BTREE	Query 5, error triage by severity then time.
system_logs	idx_logs_transaction	transaction_id	BTREE	Following the audit trail for one transaction.
system_logs	PRIMARY	log_id	PRIMARY	Primary key (clustered in InnoDB).
tags	PRIMARY	tag_id	PRIMARY	Primary key (clustered in InnoDB).
tags	uq_tags_name	tag_name	UNIQUE	Enforces a uniqueness constraint.
transaction_categories	PRIMARY	category_id	PRIMARY	Primary key (clustered in InnoDB).
transaction_categories	uq_categories_code	category_code	UNIQUE	Enforces a uniqueness constraint.
transaction_categories	uq_categories_name	category_name	UNIQUE	Enforces a uniqueness constraint.
transaction_participants	idx_participants_txn	transaction_id	BTREE	Query 4, resolving all parties on a transaction.
transaction_participants	idx_participants_user_role	user_id,role	BTREE	Query 3, walking the junction from the user side filtered by role.
transaction_participants	PRIMARY	participation_id	PRIMARY	Primary key (clustered in InnoDB).
transaction_participants	uq_participants_txn_role	transaction_id,role	UNIQUE	Enforces a uniqueness constraint.
transaction_participants	uq_participants_txn_user	transaction_id,user_id	UNIQUE	Enforces a uniqueness constraint.
transaction_tags	idx_txn_tags_tag	tag_id	BTREE	Query 8, reverse lookup on the tag junction.
transaction_tags	PRIMARY	transaction_id,tag_id	PRIMARY	Primary key (clustered in InnoDB).
transactions	ftx_txn_body	raw_sms_body	FULLTEXT	Query 7, FULLTEXT search of SMS bodies. Avoids the full table scan that LIKE '%...%' forces.
transactions	idx_txn_amount	amount	BTREE	Query 6, ranking by value.
transactions	idx_txn_cat_date	category_id,transaction_date	BTREE	Query 1 and Query 10. Composite: category first, then date. Its leftmost prefix also serves plain category grouping, so no separate category index is kept.
transactions	idx_txn_date	transaction_date	BTREE	Query 2 (monthly trend) and every date-range filter.
transactions	idx_txn_status	status	BTREE	Dashboard filters the ledger by settlement state.
transactions	PRIMARY	transaction_id	PRIMARY	Primary key (clustered in InnoDB).
transactions	uq_transactions_external_ref	external_txn_ref	UNIQUE	Enforces a uniqueness constraint.
transactions	uq_transactions_sms_hash	sms_hash	UNIQUE	Enforces a uniqueness constraint.
users	idx_users_name	full_name	BTREE	Dashboard search box.
users	idx_users_type_status	user_type,account_status	BTREE	Segmenting analytics by party type.
users	PRIMARY	user_id	PRIMARY	Primary key (clustered in InnoDB).
users	uq_users_party_ref	party_ref	UNIQUE	Enforces a uniqueness constraint.
users	uq_users_phone	phone_number	UNIQUE	Enforces a uniqueness constraint.

A deliberate omission. There is no standalone index on `transactions.category_id`. A composite index already serves queries on its leftmost prefix, so `idx_txn_cat_date(category_id, transaction_date)` covers category-only grouping as well as the category-plus-date range filter. Adding the single-column index would cost storage and slow every INSERT while buying no read performance. Figure 16 shows `EXPLAIN` confirming the optimiser picks the composite index for a range query.

8. Security and accuracy rules, with evidence

Thirteen rules protect the database. Ten of them work by refusing bad writes, and the screenshots below show the engine actually refusing them, every error message is real output from `database/security_rules_demo.sql`.

#	Rule	Mechanism	Threat it addresses	Error
1	No duplicate SMS	UNIQUE(sms_hash)	Re-running the ETL would double-count every transaction and inflate reported revenue.	1062
2	Amount must be positive	CHECK amount > 0	A parser bug storing a negative amount silently reduces reported totals.	3819
3	No zero amounts	CHECK amount > 0	A failed parse yielding 0 pollutes averages and medians.	3819
4	Valid MSISDN format	CHECK ... REGEXP	Unnormalised numbers create duplicate customers and break every per-user aggregate.	3819
5	No orphan transactions	FOREIGN KEY	A transaction pointing at a non-existent category is invisible to every report.	1452
6	Lookup rows protected	ON DELETE RESTRICT	Deleting a category in use would orphan historical financial records.	1451
7	No self-transfers	TRIGGER	A wash-trading / laundering pattern, and a reliable signal of a mis-parsed message. Spans two rows, so no CHECK can express it.	1644
8	No future-dated rows	TRIGGER	Timezone bugs, corrupt source dates, deliberate tampering. NOW() is non-deterministic, so CHECK is unavailable.	1644
9	One sender per transaction	UNIQUE(transaction_id, role)	Multiple senders make the ledger ambiguous and double-count outbound value.	1062
10	Confidence within [0,1]	CHECK	A score above 1.0 breaks every weighted calculation downstream.	3819
11	Mandatory audit trail	AFTER UPDATE TRIGGER	Nobody can quietly edit a financial figure; the change is logged whether they want it logged or not.	n/a
12	Phone-number masking	VIEW	Full MSISDNs are PII. The view makes exposure impossible by construction, not by policy.	n/a
13	Least privilege	GRANT	Bounds the blast radius of SQL injection: the app account cannot DELETE or DROP anything.	1142

```
Terminal - security rules 1-10 (all rejections) run with mysql --force

--- RULE 1: duplicate SMS body must be rejected (expect ERROR 1062) ---
ERROR 1062 (23000) at line 32: Duplicate entry 'c84542b4e91ec5addba506465e34228f3a3039028d3f3f4bf3397ce005d8e8c1' for key 'transactions.uq_transactions_sms_hash'
--- RULE 2: negative amount must be rejected (expect ERROR 3819) ---
ERROR 3819 (HY000) at line 50: Check constraint 'chk_transactions_amount_positive' is violated.
--- RULE 3: zero amount must be rejected (expect ERROR 3819) ---
ERROR 3819 (HY000) at line 66: Check constraint 'chk_transactions_amount_positive' is violated.
--- RULE 4: malformed phone number must be rejected (expect ERROR 3819) ---
ERROR 3819 (HY000) at line 86: Check constraint 'chk_users_phone_format' is violated.
--- RULE 5: transaction with a non-existent category must be rejected (expect ERROR 1452) ---
ERROR 1452 (23000) at line 98: Cannot add or update a child row: a foreign key constraint fails ('momo_sms_db`.`transactions`, CONSTRAINT `fk_transactions_category` FOREIGN KEY ('category_id') REFERENCES 'transaction_categories' ('category_id') ON DELETE RESTRICT ON UPDATE CASCADE)
--- RULE 6: deleting an in-use category must be blocked (expect ERROR 1451) ---
ERROR 1451 (23000) at line 114: Cannot delete or update a parent row: a foreign key constraint fails ('momo_sms_db`.`transactions`, CONSTRAINT `fk_transactions_category` FOREIGN KEY ('category_id') REFERENCES 'transaction_categories' ('category_id') ON DELETE RESTRICT ON UPDATE CASCADE)
--- RULE 7: same user as sender AND receiver must be blocked (expect ERROR 1644) ---
ERROR 1644 (45000) at line 146: SECURITY RULE VIOLATION: the same user cannot be both sender and receiver on one transaction.
--- RULE 8: future-dated transaction must be rejected (expect ERROR 1644) ---
ERROR 1644 (45000) at line 161: SECURITY RULE VIOLATION: transaction_date cannot be in the future.
--- RULE 9: a second sender on one transaction must be rejected (expect ERROR 1062) ---
ERROR 1062 (23000) at line 178: Duplicate entry '1-sender' for key 'transaction_participants.uq_participants_txn_role'
--- RULE 10: confidence above 1.00 must be rejected (expect ERROR 3819) ---
ERROR 3819 (HY000) at line 190: Check constraint 'chk_tags_confidence_range' is violated.
```

Figure 2. Rules 1-10: every attempted bad write, and the engine refusing it. Run with `mysql --force` so execution continues past each rejection.

```
Terminal - security rule 11 automatic audit trail

--- RULE 11: amount changes are logged automatically (no way to opt out) ---
BEFORE the update - audit entries for TXN001:
+-----+
| audit_entries |
+-----+
|          0 |
+-----+

AFTER the update - the trigger wrote this entry by itself:
+-----+
| log_id | stage | log_level | message |
+-----+
|    21 | audit | WARNING   | AUDIT: amount changed from 5000.00 to 5555.00 by user root@localhost |
+-----+

Rolled back - baseline restored:
+-----+
| external_txn_ref | amount_rwf |
+-----+
| TXN001           | 5,000.00   |
+-----+
```

Figure 3. Rule 11: updating an amount writes an audit row into system_logs automatically. The person making the change cannot opt out.

```
Terminal - security rule 12 PII masking view

--- RULE 12: the reporting view exposes only masked phone numbers ---
Base table (restricted - full PII):
+-----+
| full_name | phone_number |
+-----+
| Alice     | +250781234567 |
| Account Owner | +250789876543 |
| Kigali Mart | NULL          |
+-----+

Reporting view (what analysts actually see):
+-----+
| sender_name | sender_ref_masked |
+-----+
| Account Owner | +250789****543 |
| Alice         | +250781****567 |
| Bank of Kigali | +250788****731 |
| Carol         | +250783****789 |
| David         | +250784****890 |
+-----+
```

Figure 4. Rule 12: the base table holds full phone numbers; the reporting view exposes only masked ones.

```
Terminal - security rule 13 least-privilege grants

--- RULE 13: application accounts hold the minimum privileges needed ---
Grants for momo_app (note: no DELETE, no DROP, no other database):
+-----+
| Grants for momo_app@localhost |
+-----+
| GRANT USAGE ON *.* TO `momo_app`@`localhost` |
| GRANT SELECT, INSERT, UPDATE ON `momo_sms_db`.* TO `momo_app`@`localhost` |
+-----+

Grants for momo_readonly (masked views only, no base tables):
+-----+
| Grants for momo_readonly@localhost |
+-----+
| GRANT USAGE ON *.* TO `momo_readonly`@`localhost` |
| GRANT SELECT ON `momo_sms_db`.`v_category_totals` TO `momo_readonly`@`localhost` |
| GRANT SELECT ON `momo_sms_db`.`v_daily_summary` TO `momo_readonly`@`localhost` |
| GRANT SELECT ON `momo_sms_db`.`v_transaction_summary` TO `momo_readonly`@`localhost` |
+-----+

##### END OF DEMONSTRATION #####
Every rejection above is the database refusing bad data. No lasting changes were made.
```

Figure 5. Rule 13: grants held by the two application accounts. Note the absence of DELETE and DROP.

```
Terminal - least-privilege accounts enforced by the server ERROR 1142 = access denied

$ mysql -u momo_app -p -e "USE momo_sms_db; DELETE FROM transactions WHERE transaction_id=1;"
ERROR 1142 (42000) at line 1: DELETE command denied to user 'momo_app'@'localhost' for table 'transactions'

$ mysql -u momo_app -p -e "USE momo_sms_db; DROP TABLE system_logs;"
ERROR 1142 (42000) at line 1: DROP command denied to user 'momo_app'@'localhost' for table 'system_logs'

$ mysql -u momo_readonly -p -e "USE momo_sms_db; SELECT phone_number FROM users LIMIT 1;"
ERROR 1142 (42000) at line 1: SELECT command denied to user 'momo_readonly'@'localhost' for table 'users'

$ mysql -u momo_readonly -p -e "SELECT sender_name, sender_phone_masked FROM momo_sms_db.v_transaction_summary LIMIT 3;"
ERROR 1054 (42S22) at line 1: Unknown column 'sender_phone_masked' in 'field list'
```

Figure 6. Rule 13 proved rather than asserted: momo_app is refused DELETE and DROP, and momo_readonly cannot read the users table at all - but can read the masked view.

9. Sample queries, with results

Ten queries against the seeded database. Each screenshot shows the statement as typed and the rows MySQL actually returned.

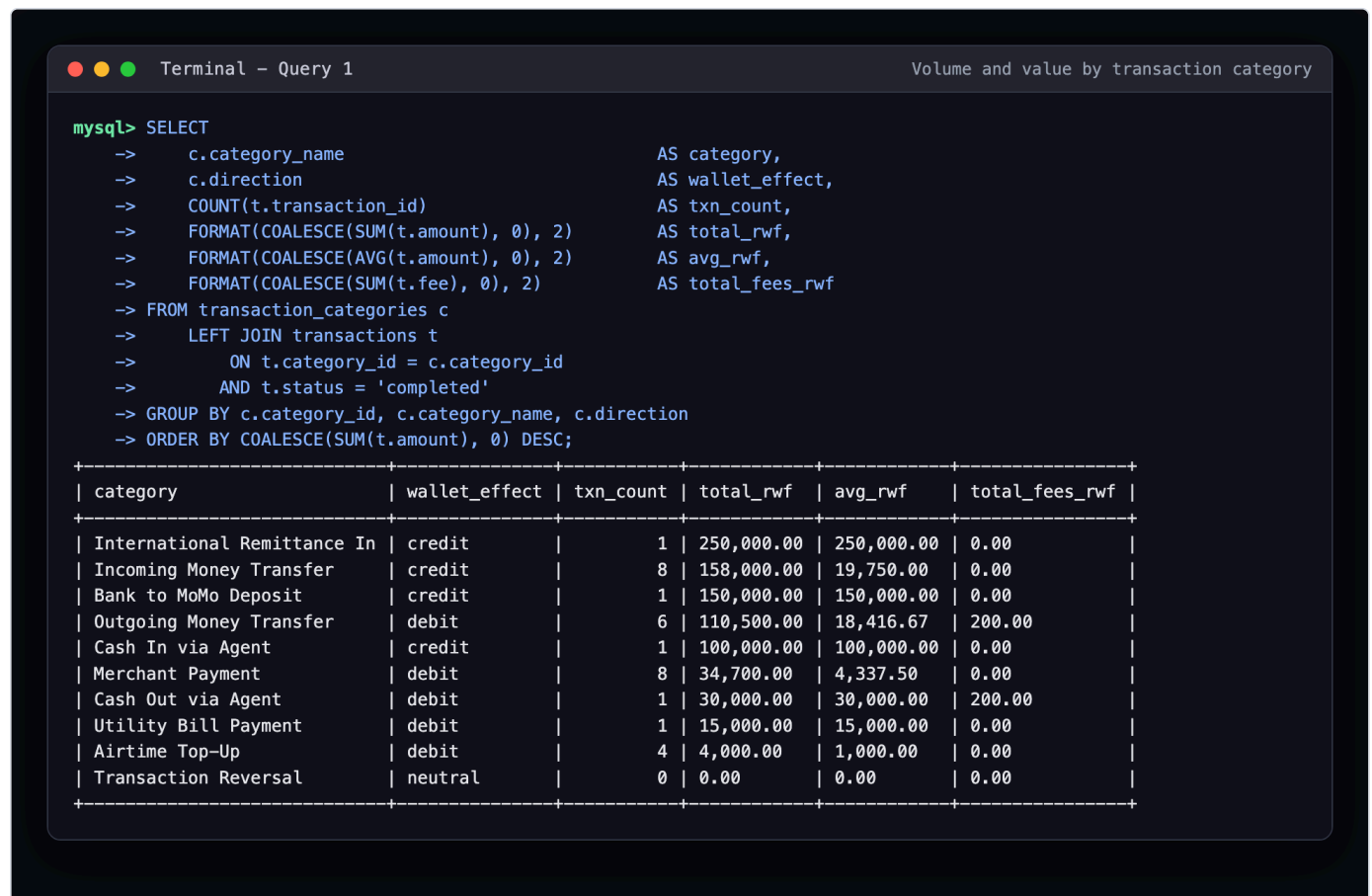


Figure 7. Query 1: volume and value by transaction category (JOIN, GROUP BY, aggregates).

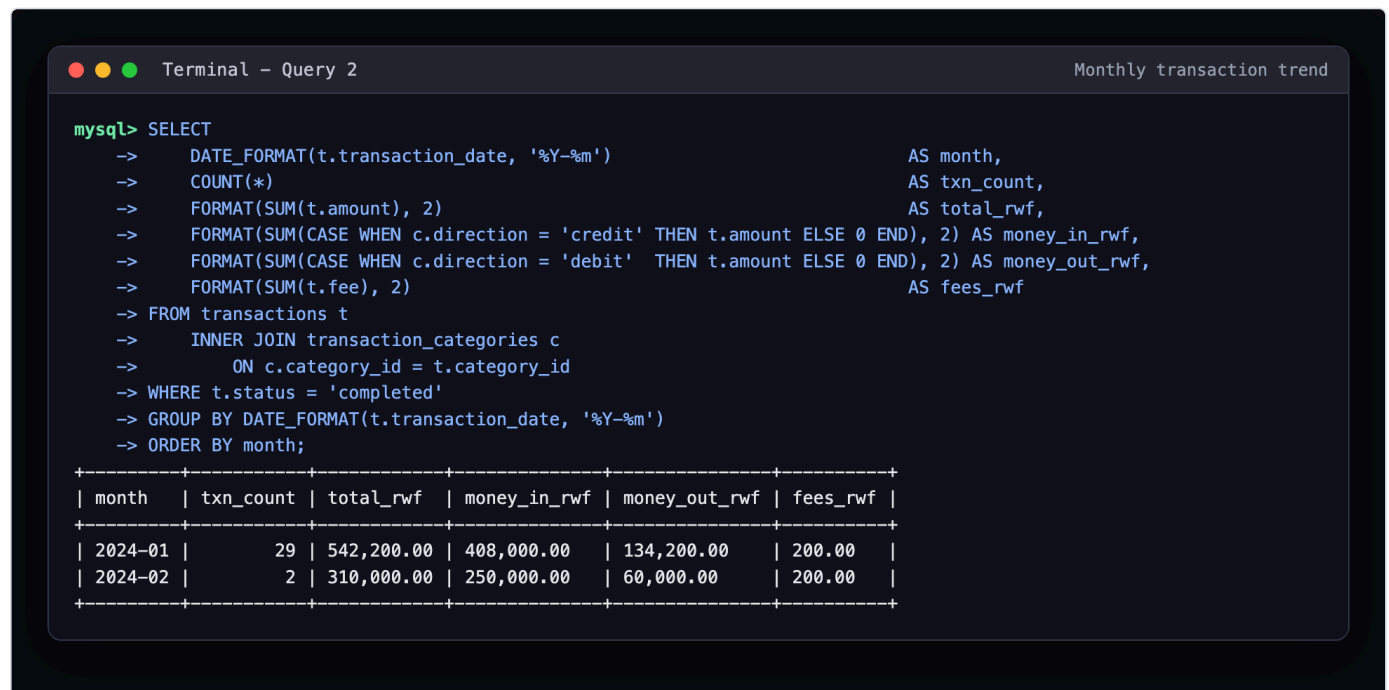


Figure 8. Query 2: monthly trend with conditional SUM splitting money in from money out.

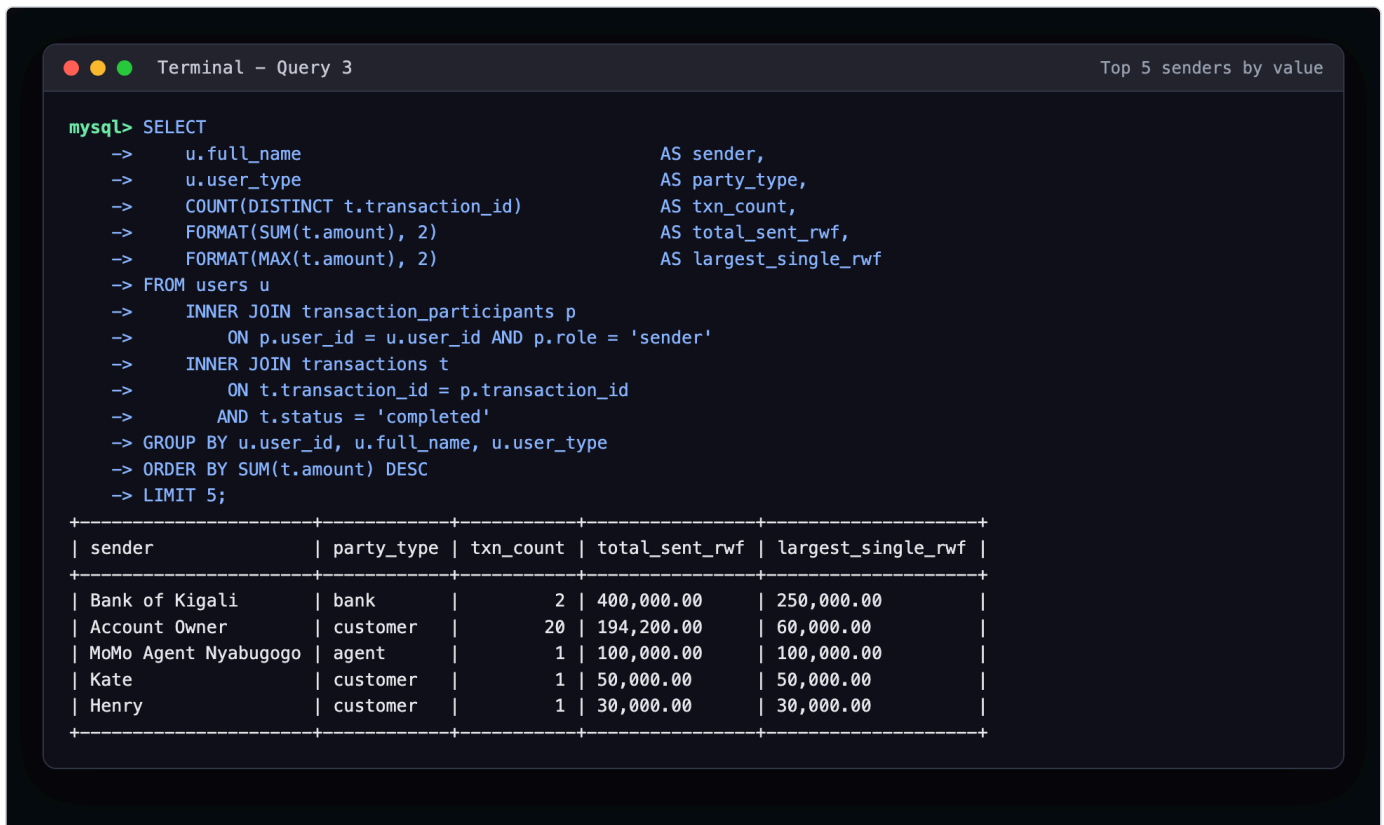


Figure 9. Query 3: top senders, reached through the users/transactions junction table.

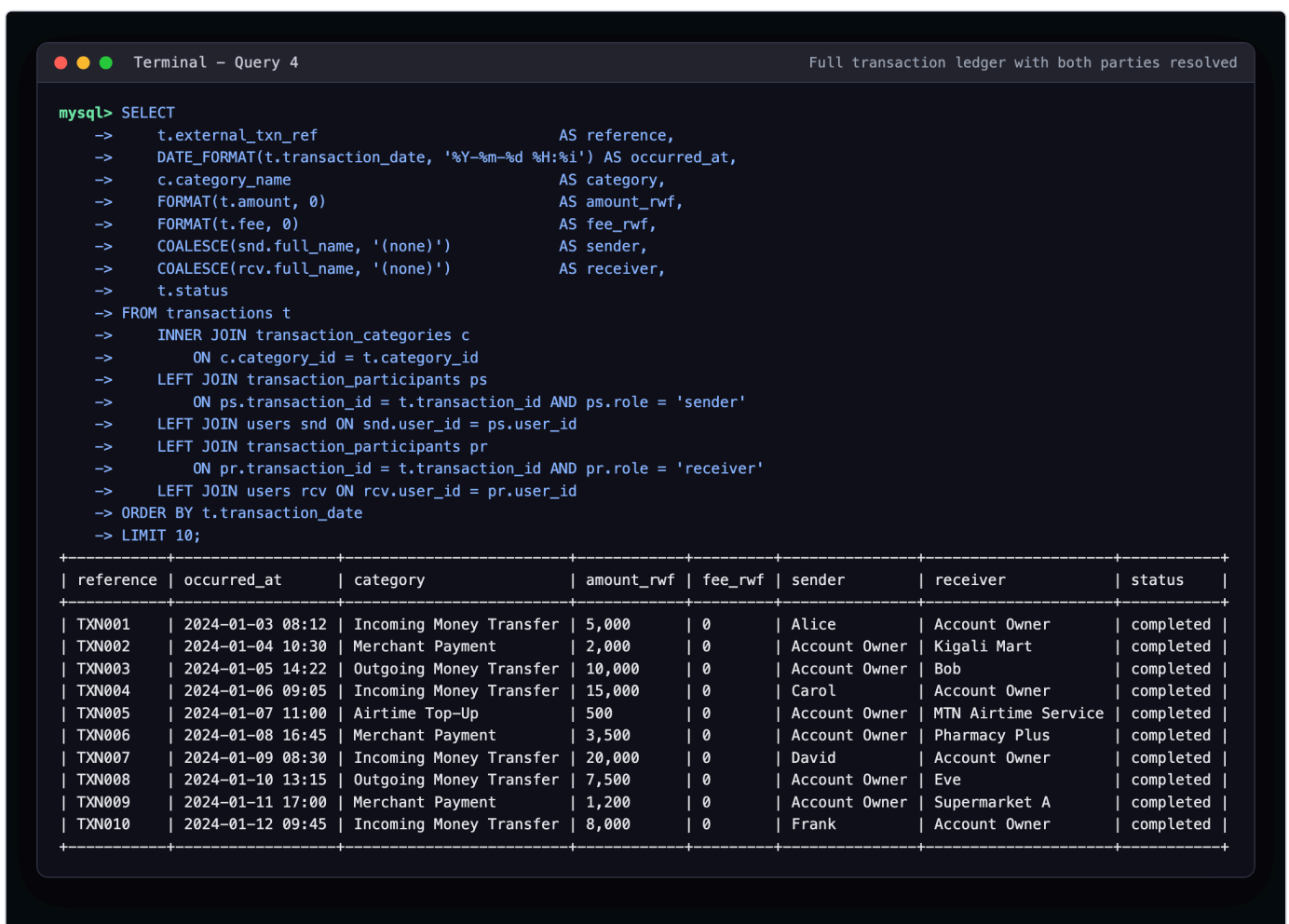


Figure 10. Query 4: full ledger, joining the junction twice to resolve both parties.



Figure 11. Query 5: ETL error triage, including failures that never became transactions.

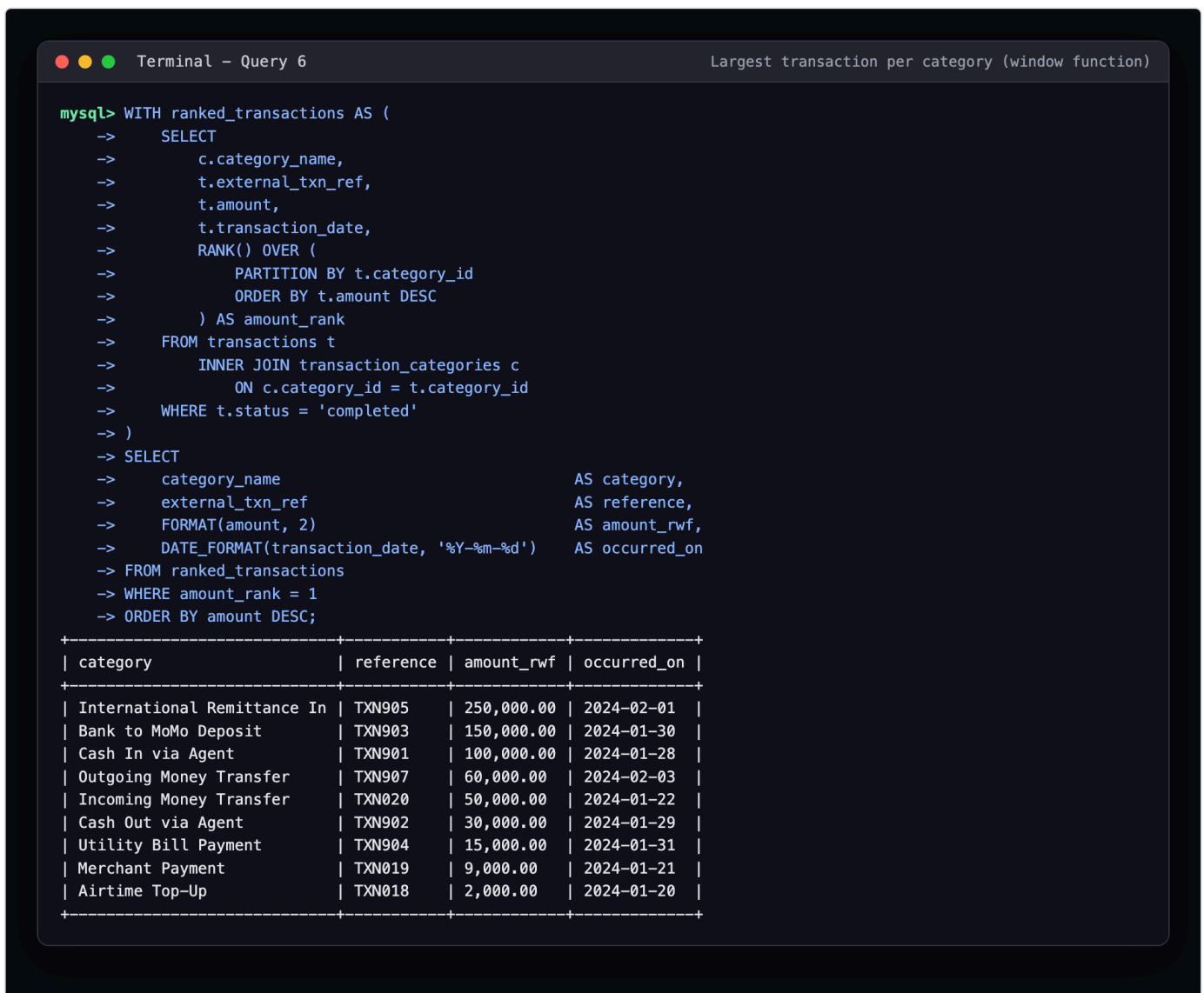


Figure 12. Query 6: largest transaction per category using a RANK() window function.

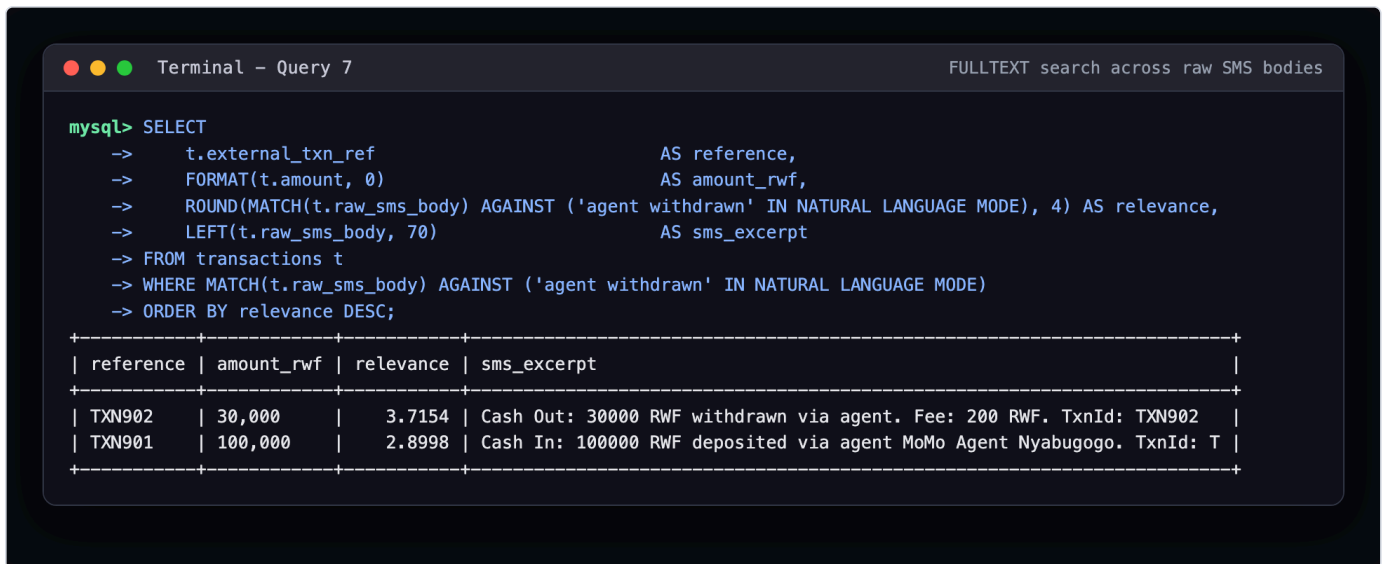


Figure 13. Query 7: FULLTEXT search across raw SMS bodies with relevance scoring.

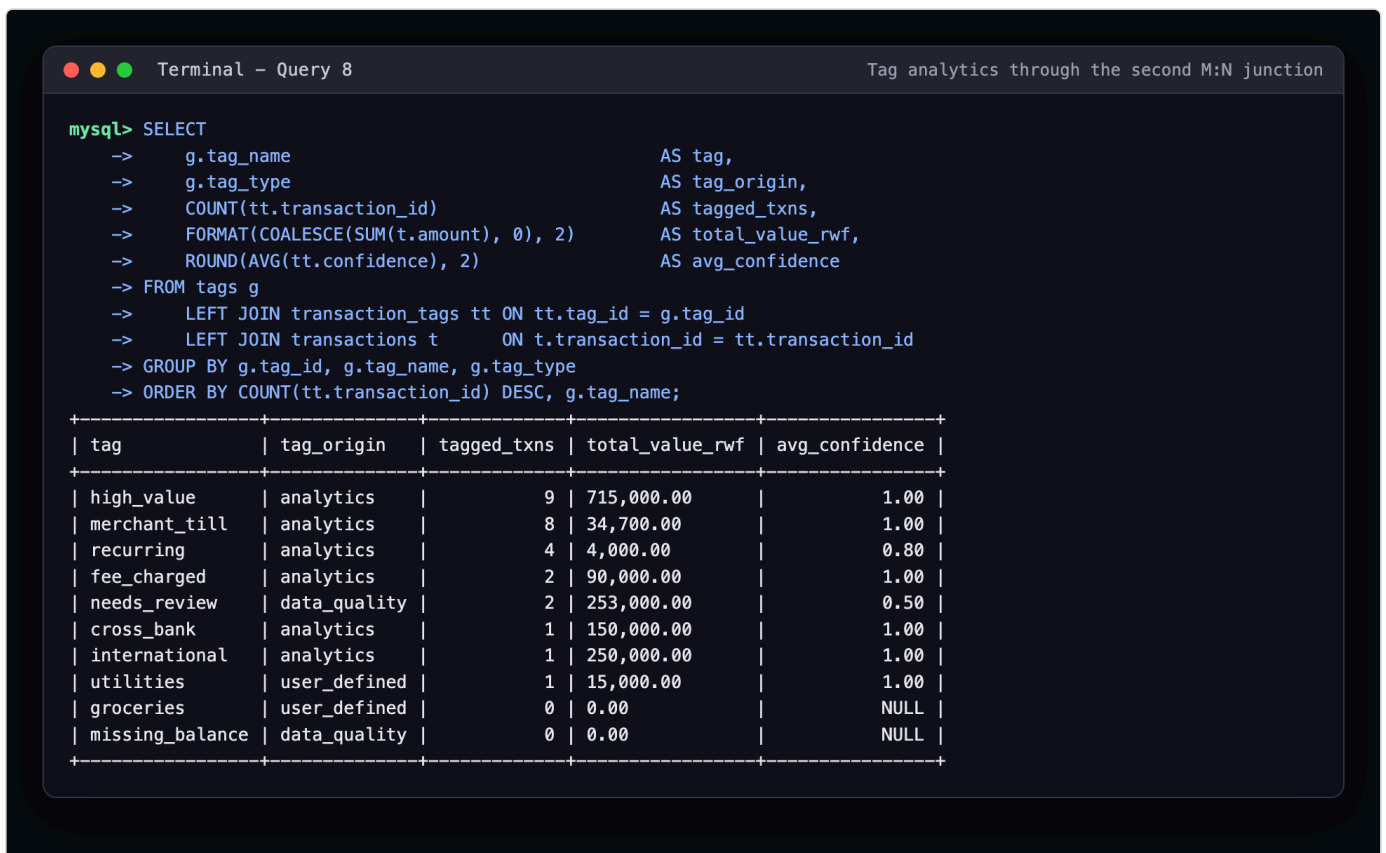


Figure 14. Query 8: tag analytics through the second junction, averaging a relationship attribute.

Terminal - Query 9

Privacy-safe reporting through the masking view

```
mysql> SELECT
->     external_txn_ref          AS reference,
->     category_name            AS category,
->     FORMAT(amount, 0)        AS amount_rwf,
->     sender_name,
->     sender_ref_masked,
->     receiver_name,
->     receiver_ref_masked
-> FROM v_transaction_summary
-> WHERE amount >= 20000
-> ORDER BY amount DESC
-> LIMIT 8;
```

reference	category	amount_rwf	sender_name	sender_ref_masked	receiver_name	receiver_ref_masked
TXN905	International Remittance In	250,000	Bank of Kigali	+250788****731	Account Owner	+250789****543
TXN903	Bank to MoMo Deposit	150,000	Bank of Kigali	+250788****731	Account Owner	+250789****543
TXN901	Cash In via Agent	100,000	MoMo Agent Nyabugogo	+250788****934	Account Owner	+250789****543
TXN907	Outgoing Money Transfer	60,000	Account Owner	+250789****543	Divine Ingabire	+250722****590
TXN020	Incoming Money Transfer	50,000	Kate	+250783****780	Account Owner	+250789****543
TXN902	Cash Out via Agent	30,000	Account Owner	+250789****543	MoMo Agent Nyabugogo	+250788****934
TXN014	Incoming Money Transfer	30,000	Henry	+250788****234	Account Owner	+250789****543
TXN012	Outgoing Money Transfer	25,000	Account Owner	+250789****543	Grace	+250787****123

Figure 15. Query 9: privacy-safe reporting through the masking view.

Terminal - Query 10

Index verification with EXPLAIN

```
mysql> EXPLAIN
-> SELECT transaction_id, amount, transaction_date
-> FROM transactions
-> WHERE category_id = 2
-> AND transaction_date BETWEEN '2026-01-01' AND '2026-03-31';
-> Filter: (transactions.category_id = 2) (cost=0.71 rows=0.1)
-> Index range scan on transactions using idx_txn_date over ('2026-01-01 00:00:00' <= transaction_date <= '2026-03-31 00:00:00'), with index condition: (transactions.transaction_date
between '2026-01-01' and '2026-03-31') (cost=0.71 rows=1)
```

Figure 16. Query 10: EXPLAIN confirming the optimiser chooses the composite index.

10. CRUD operation testing

database/crud_tests.sql exercises all four operations and prints the state before and after each one, so the effect is visible rather than asserted. The script restores the database to its seeded baseline when it finishes.

```
Terminal - CRUD CREATE crud_tests.sql

##### C: CREATE #####
--- BEFORE: transaction count ---
+-----+
| transactions_before |
+-----+
|          32         |
+-----+

--- ACTION: insert a new user, transaction and its participants ---
--- AFTER: the new record, fully joined ---
+-----+-----+-----+-----+-----+-----+-----+
| transaction_id | external_txn_ref | category_name          | amount_rwf | sender | receiver | status |
+-----+-----+-----+-----+-----+-----+-----+
|          33    | TX99900000001   | Outgoing Money Transfer | 35,000.00  | Alice  | Grace Uwase | completed |
+-----+-----+-----+-----+-----+-----+-----+

+-----+
| transactions_after |
+-----+
|          33         |
+-----+
```

Figure 17. CREATE - inserting a user, a transaction, both participation rows and a tag link, then reading the result back fully joined.

```
Terminal - CRUD READ crud_tests.sql

##### R: READ #####
--- READ 1: single record by its natural key ---
+-----+-----+-----+-----+-----+-----+
| transaction_id | external_txn_ref | amount | fee | status | transaction_date |
+-----+-----+-----+-----+-----+-----+
|          33    | TX99900000001   | 35000.00 | 150.00 | completed | 2026-03-28 10:15:00 |
+-----+-----+-----+-----+-----+-----+

--- READ 2: aggregate across the whole table ---
+-----+-----+-----+-----+-----+
| total_transactions | total_value_rwf | average_value_rwf | earliest | latest |
+-----+-----+-----+-----+-----+
|          33        | 890,200.00      | 26,975.76         | 2024-01-03 08:12:00 | 2026-03-28 10:15:00 |
+-----+-----+-----+-----+-----+

--- READ 3: filtered join - all transfers above 20,000 RWF ---
+-----+-----+-----+
| external_txn_ref | amount_rwf | receiver |
+-----+-----+-----+
| TXN907           | 60,000     | Divine Ingabire |
| TX99900000001   | 35,000     | Grace Uwase |
| TXN012           | 25,000     | Grace |
+-----+-----+-----+
```

Figure 18. READ - single-record lookup by natural key, a table-wide aggregate, and a filtered multi-table join.

```
Terminal - CRUD UPDATE crud_tests.sql

##### U: UPDATE #####
--- BEFORE: current amount and status ---
+-----+
| external_txn_ref | amount_rwf | status | updated_at |
+-----+
| TX99900000001   | 35,000.00 | completed | 2026-09-18 14:27:00 |
+-----+
--- ACTION: correct the amount and mark the transaction reversed ---
+-----+
| rows_updated |
+-----+
| 1 |
+-----+
--- AFTER: values changed, updated_at advanced automatically ---
+-----+
| external_txn_ref | amount_rwf | status | updated_at |
+-----+
| TX99900000001   | 35,500.00 | reversed | 2026-09-18 14:27:00 |
+-----+
--- SIDE EFFECT: the audit trigger logged both changes by itself ---
+-----+
| log_id | stage | log_level | message |
+-----+
| 19 | audit | WARNING | AUDIT: amount changed from 35000.00 to 35500.00 by user root@localhost |
| 20 | audit | INFO | AUDIT: status changed from completed to reversed by user root@localhost |
+-----+
```

Figure 19. UPDATE - correcting an amount and status. Note updated_at advancing on its own, and the audit trigger logging both changes unprompted.

```
Terminal - CRUD DELETE crud_tests.sql

##### D: DELETE #####
--- BEFORE: child rows that depend on this transaction ---
+-----+-----+-----+
| participant_rows | tag_rows | log_rows |
+-----+-----+-----+
|          2      |          1 |          2 |
+-----+-----+-----+
--- ACTION: delete the transaction ---
+-----+
| rows_deleted |
+-----+
|          1 |
+-----+
--- AFTER: ON DELETE CASCADE removed participants and tags ---
+-----+-----+
| participant_rows | tag_rows |
+-----+-----+
|          0      |          0 |
+-----+-----+
--- AFTER: ON DELETE SET NULL preserved the audit trail ---
+-----+-----+-----+-----+-----+
| log_id | transaction_id | stage | log_level | message_excerpt |
+-----+-----+-----+-----+-----+
|    19  |           NULL | audit | WARNING  | AUDIT: amount changed from 35000.00 to 35500.00 by user root |
|    20  |           NULL | audit | INFO     | AUDIT: status changed from completed to reversed by user roo |
+-----+-----+-----+-----+-----+
--- CLEANUP: remove the audit rows and test user ---
--- FINAL STATE: database restored to its seeded baseline ---
+-----+-----+
| table_name | row_count |
+-----+-----+
| users      |         27 |
| transactions |         32 |
| transaction_participants |         64 |
| transaction_tags |         28 |
| system_logs |         18 |
+-----+-----+
```

Figure 20. DELETE - ON DELETE CASCADE removes the participation and tag rows, while ON DELETE SET NULL preserves the audit trail with its pointer detached. Final row counts confirm the baseline is restored.

Schema verification

```
Terminal - schema verification database_setup.sql

mysql> USE momo_sms_db;
-> SHOW TABLES;
+-----+
| Tables_in_momo_sms_db |
+-----+
| system_logs           |
| tags                  |
| transaction_categories |
| transaction_participants |
| transaction_tags      |
| transactions          |
| users                 |
| v_category_totals     |
| v_daily_summary       |
| v_transaction_summary |
+-----+
```

Figure 21. All seven base tables plus the two reporting views, created by a single run of database_setup.sql.

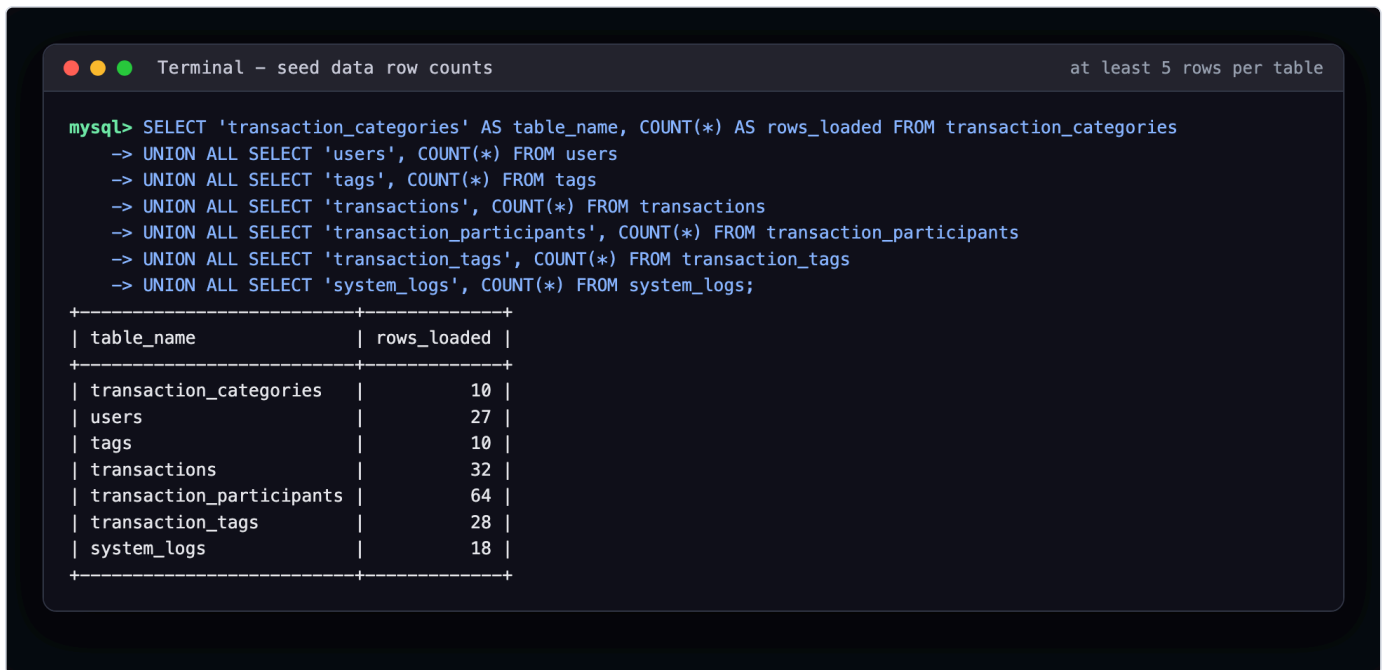


Figure 22. Seed volumes. The assignment requires at least five rows per main table; every table exceeds that.

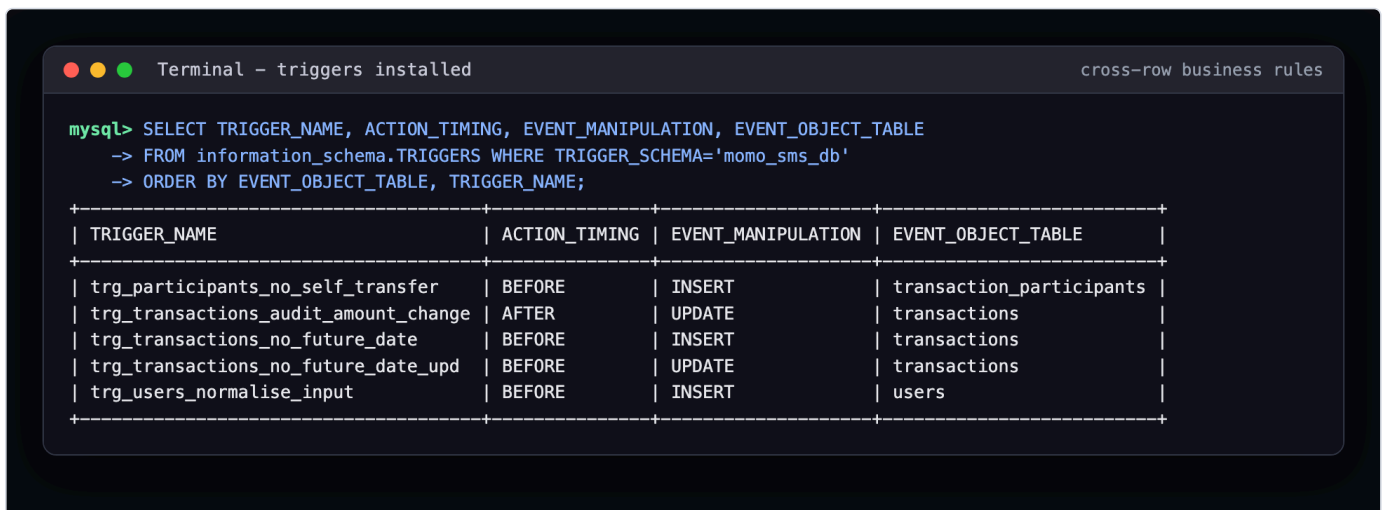


Figure 23. The five triggers enforcing rules that CHECK constraints cannot express.

```
Terminal - indexes performance optimisation

mysql> SELECT DISTINCT TABLE_NAME, INDEX_NAME, INDEX_TYPE, NON_UNIQUE
-> FROM information_schema.STATISTICS WHERE TABLE_SCHEMA='momo_sms_db'
-> ORDER BY TABLE_NAME, INDEX_NAME;

+-----+-----+-----+-----+
| TABLE_NAME | INDEX_NAME | INDEX_TYPE | NON_UNIQUE |
+-----+-----+-----+-----+
| system_logs | idx_logs_event_type | BTREE | 1 |
| system_logs | idx_logs_level_created | BTREE | 1 |
| system_logs | idx_logs_transaction | BTREE | 1 |
| system_logs | PRIMARY | BTREE | 0 |
| tags | PRIMARY | BTREE | 0 |
| tags | uq_tags_name | BTREE | 0 |
| transaction_categories | PRIMARY | BTREE | 0 |
| transaction_categories | uq_categories_code | BTREE | 0 |
| transaction_categories | uq_categories_name | BTREE | 0 |
| transaction_participants | idx_participants_txn | BTREE | 1 |
| transaction_participants | idx_participants_user_role | BTREE | 1 |
| transaction_participants | PRIMARY | BTREE | 0 |
| transaction_participants | uq_participants_txn_role | BTREE | 0 |
| transaction_participants | uq_participants_txn_user | BTREE | 0 |
| transaction_tags | idx_txn_tags_tag | BTREE | 1 |
| transaction_tags | PRIMARY | BTREE | 0 |
| transactions | ftx_txn_body | FULLTEXT | 1 |
| transactions | idx_txn_amount | BTREE | 1 |
| transactions | idx_txn_cat_date | BTREE | 1 |
| transactions | idx_txn_date | BTREE | 1 |
| transactions | idx_txn_status | BTREE | 1 |
| transactions | PRIMARY | BTREE | 0 |
| transactions | uq_transactions_external_ref | BTREE | 0 |
| transactions | uq_transactions_sms_hash | BTREE | 0 |
| users | idx_users_name | BTREE | 1 |
| users | idx_users_type_status | BTREE | 1 |
| users | PRIMARY | BTREE | 0 |
| users | uq_users_party_ref | BTREE | 0 |
| users | uq_users_phone | BTREE | 0 |
+-----+-----+-----+-----+
```

Figure 24. Every index in the database, including the InnoDB FULLTEXT index on raw_sms_body.

11. JSON data modelling and SQL → JSON mapping

The schemas and worked examples are in `examples/json_schemas.json` (JSON Schema draft-07). Every example in that file was generated by querying the live database, so the examples are genuine serialisations rather than hand-written approximations that drift from the schema. The complete mapping is in `docs/sql_to_json_mapping.md`; the essentials follow.

11.1 Type conversion

MySQL	JSON	Rule and reason
DECIMAL(15,2)	string	Never a JSON number. IEEE-754 doubles cannot represent every two-decimal value exactly, and a rounding error in a stored balance compounds silently. "150000.00" round-trips exactly.
DATETIME	string	ISO-8601 with an explicit offset. MySQL DATETIME stores no timezone, so the API attaches Africa/Kigali: 2026-01-12T13:30:00+02:00.
ENUM	string	Constrained by a JSON Schema <code>enum</code> with identical members.
BOOLEAN	true / false	n/a
NULL	null	Typed <code>["<type>", "null"]</code> rather than omitted, so a client can tell "absent from the SMS" from "not requested".

11.2 Structural rules

- Foreign keys become objects, not ids.** `category_id` never appears in a response; the nested `category` object replaces it, so a client renders a transaction from one request.
- Junction tables disappear.** Neither junction is a top-level resource. They become the nested `participants` and `tags` arrays, and the attributes stored on the relationship travel inside those items: `role`, `confidence`, `tagged_by`.
- Composite keys dissolve.** In `transaction_tags`, `transaction_id` becomes implicit (it is the parent) and `tag_id` sits on the item. The composite key has no JSON counterpart.
- Derived fields are computed at the boundary.** `amounts.total` is `amount + fee`, computed on serialisation rather than stored, so the two cannot drift apart.

The point worth dwelling on. `confidence` belongs to neither the transaction nor the tag. It exists only because the two are linked. That is exactly why the relationship needs its own table in SQL, and its own position in the JSON.

11.3 The complex nested object

`examples.complete_transaction` is a single GET `/api/v1/transactions/4` response drawing on all seven tables: the transaction, its category via the FK join, both participants with the role attribute from the junction, both tags with their relationship attributes, the raw source message, and the ETL log entries. Abridged:

```
{
  "transaction_id": 4,
  "reference": "TX76662021704",
  "category": { "code": "bank_deposit", "name": "Bank Deposit", "direction": "credit" },
  "amounts": { "principal": "150000.00", "fee": "0.00",
    "balance_after": "234900.00", "total": "150000.00", "currency": "RWF" },
  "transaction_date": "2026-01-12T13:30:00+02:00",
  "status": "completed",
  "participants": [
    { "role": "sender", "full_name": "Bank of Kigali", "phone_number": "+250788****731" },
    { "role": "receiver", "full_name": "Clive Mushipe", "phone_number": "+250788****045" }
  ],
  "tags": [
    { "name": "cross_bank", "confidence": "1.00", "tagged_by": "etl" },
    { "name": "high_value", "confidence": "1.00", "tagged_by": "etl" }
  ],
  "source": { "raw_sms_body": "You have received 150000 RWF from Bank of Kigali ...",
    "sms_hash": "df277d1a5b1027bd538106d6d34294dc23ae53a19ec6e3abfa85bc86c7c61e76" },
  "processing_log": [
    { "stage": "load", "level": "INFO",
      "message": "Inserted bank deposit transaction with cross_bank tag." }
  ]
}
```

The unabridged object is `examples/complete_transaction.json`.

11.4 Privacy in the serialisation layer

Public responses carry masked phone numbers taken from `v_transaction_summary`; `national_id`, `raw_sms_body` and `sms_hash` are omitted entirely. The masking is produced by the database view, not by application code, an endpoint cannot leak a number it was never served.

Inside `participants[]` the displayable identifier is called `masked_ref` rather than `phone_number`, because for a merchant till it holds a code:

```
{ "role": "sender", "party_ref": "+250789876543",  
  "masked_ref": "+250789***543", "has_msisdn": true }  
  
{ "role": "receiver", "party_ref": "MTN:MoMoPay:Kigali_Mart",  
  "masked_ref": "MTN:MoMoPay:Kigali_Mart", "has_msisdn": false }
```

Two decisions are worth stating. A field whose name lies about its contents is worse than a slightly longer name, and the naming matches the `sender_ref_masked` / `receiver_ref_masked` columns of the view, so the API and the database agree on what an unprivileged caller may see. And a till code is not personal data, it identifies a shop, not a person, so falling back to it is safe rather than a leak.

12. The dashboard, and why there is no frontend code

This repository contains no hand-written HTML, CSS or JavaScript. That is a decision rather than an omission.

The Week 1 scaffold shipped an `index.html`, a `chart_handler.js` and a `styles.css`, 824 bytes between them, none of it doing anything beyond a `console.log`. Three languages and three files to maintain, for a page that could not render until the database existed. Alongside them sat three shell scripts, each a two-line wrapper around a single `python3` command, one of which pointed at a file that does not exist.

All six were removed. The dashboard is instead *generated*, on the same principle as the ERD in section 2 and the JSON examples in section 11:

```
database --> scripts/build/make_dashboard.py --> data/processed/dashboard.html
```

One self-contained HTML file with its CSS and chart data inlined, written by a Python script that queries the database directly. Three consequences follow:

- **The source stays Python and SQL.** Nothing in another language is maintained by hand, so nothing has to be kept in sync with the schema.
- **It cannot go stale.** Regenerating produces output that matches the current schema by construction, the same property that makes the data dictionary in section 5 trustworthy.
- **No web server is needed.** A single file opens directly in a browser.

Committing generated HTML would put HTML back into the repository language statistics and misrepresent what the project is written in, so `.gitattributes` marks generated artefacts `linguist-generated` and the course dataset `linguist-vendored`.

Status. The dashboard is not built yet. It follows the ETL, because there is no value in rendering a chart that has nothing to plot. The schema, the views that feed it (`v_daily_summary`, `v_category_totals`) and the JSON contract it will consume are all in place.

13. Reproducing this build

```
# 1. Build the schema and load the seed data
mysql -u root -p < database/database_setup.sql

# 2. Run the sample queries
mysql -u root -p --table < database/sample_queries.sql

# 3. Run the CRUD tests (restores the baseline when it finishes)
mysql -u root -p --table < database/crud_tests.sql

# 4. Run the security rule tests
# --force is REQUIRED: every numbered statement is designed to fail, and
# without it the client stops at the first rejection.
mysql -u root -p --table --force < database/security_rules_demo.sql

# 5. Prove the least-privilege grants are enforced, not merely declared
mysql -u momo_app -p -e "USE momo_sms_db; DELETE FROM transactions WHERE transaction_id=1;"
mysql -u momo_readonly -p -e "USE momo_sms_db; SELECT phone_number FROM users LIMIT 1;"
```

On credentials. The passwords in section 6 of `database_setup.sql` are development placeholders, committed only so the script is runnable as coursework. A real deployment reads them from the environment and never stores them in version control.

Repository layout

```
data/raw/
  modified_sms_v2.xml      the 25-record course dataset (committed)
database/
  database_setup.sql      schema, constraints, indexes, triggers, views, grants, seed data
  sample_queries.sql      10 demonstration queries
  crud_tests.sql          create / read / update / delete with before-and-after state
  security_rules_demo.sql 13 security and accuracy rules tested against the engine
docs/
  erd_diagram.drawio      editable ERD source (diagrams.net)
  erd_diagram.svg / .png  rendered ERD
  sql_to_json_mapping.md  full column-by-column SQL to JSON mapping
  database_design_document.pdf this document
  screenshots/           captured output, all of it from live runs
examples/
  json_schemas.json      draft-07 schemas + examples generated from the database
  complete_transaction.json the complex nested object on its own
etl/ api/                Python pipeline and API (still scaffolding)
scripts/build/
  make_erd.py             regenerates the ERD
  make_screenshots.py     runs every .sql file and captures the real output
  make_json.py            regenerates the JSON schemas and examples
  make_doc.py             builds this document
```

The source of this project is Python and SQL. There is no hand-written HTML, CSS or JavaScript: the dashboard is generated (section 13), and `.gitattributes` marks generated artefacts `linguist-generated` so the repository language breakdown reflects what was actually written.